IT UNIVERSITY OF COPENHAGEN

Functional Programming

Jesper Bengtson

Patrick Bahr

Who are we?

Jesper Bengtson

- Associate professor at ITU since 2013
- Member of the Programming, Logics and Semantics research group
- Researches formal verification of software
  - Proof assistants
  - Distributed systems
  - Message-passing languages

Who are we?

Patrick Bahr

- At ITU since 2015
- Member of the Programming, Logics and Semantics research group
- Research
  - functional programming languages
  - type theory & formal verification
  - compilers

Who are we?

We have two groups of talented and friendly TAs that will help you throughout the semester

- Exercise sessions (Wednesdays 12-14 in 2A12-14, 2A52, 2A54)
  - Assignments, help with your code, tech support
- Live Programming (Wednesdays 14-16 in Aud 1)
  - Examples, good practices, Kattis, Exam solutions, ...

Who are we?

Teaching assistants (Exercise sessions)

- Adam Hadou Temsamani email: ahad@itu.dk
- David Martin Sørensen email: daso@itu.dk
- Lasse Faurby Klausen email: niklac@itu.dk
- Niklas Christensen email: lafakl@itu.dk
- Sebastian Cloos Hylander email: admsehy@itu.dk

Who are we?

Teaching assistants (Live Coding)

- Emilia Helsted email: emihel@itu.dk
- Theis Per Holm email: admthph@itu.dk
- Sebastian Cloos Hylander email: admsehy@itu.dk

Lectures and exercises

- **Assignment for the coming week**: Mondays 08:00
- **Lectures**: Tuesdays, 12:15-14:00, Aud1
- **Exercise sessions**: Wednesday, 12:15-1400,
  2A12-14, 2A52, 2A54
- **Live Coding**: Wednesday, 14:15-16:00, Aud1
- **Hand-ins**: Mondays by 08:00

Textbook

Functional Programming using F#

- Written by Michael R. Hansen and Hans Rischel
- ISBN: 9781107019027
- http://www2.imm.dtu.dk/~mire/FSharpBook/
- Read the relevant chapters in advance of lectures!

Functional programming

Functional Programming Paradigm

- First class functions
- Higher-order functions
- Type inference and polymorphism
- Recursion and tail recursion
- Algebraic data types
- Strict and lazy evaluation

Functional programming

Memory management

- Garbage collection
- Reference types
- Mutable vs immutable data

Parallel programming

- Divide and conquer

Using F#

- F# is an open-source functional language integrated in the Visual Studio development platform.
- Access to all of the .NET libraries
- Available for Windows, Linux, and MacOS (https://fsharp.org/)

Mandatory activities

- 10 weekly **individual** assignments worth 2 points which together will have you writing an interpreter for a small imperative language.
- You must get 16 points to be eligible to attend the exam
- If you are eligible for the exam from a previous year, you are still eligible for this exam.

Colour coded

- Assignments are colour coded
  - green Basic exercises. Recommended if you're aiming to just pass the course.
  - yellow Intermediate exercises. Recommended if you are aiming for a decent grade on the course
  - red Advanced exercises. Recommended if you are aiming for a high grade on the course.
- We highly recommend going for the yellow or red assignment.

Individual weekly assignments

- Due Mondays at 08:00
- Graded the day after (0-2 points)
- Y… …n the n… …oints
- A… …be g… …**.**
- Assignments will be handed in using CodeGrade.
- **Work individually**

Some deadlines are pushed a week to not collide with the hand-ins with the second-year project

Individual weekly assignments

- Due Mondays at 08:00

You have two attempts for each assignment. As long as you have made serious work on one of these attempts (handing in something that nearly works) then you will be granted an extension to resubmit towards the end of the course

If you do not hand in, or hand in something that is barely working you will not get this resubmission attempt

The TAs will let you know immediately if you have been granted an extension or not

- **Work individually**

Individual weekly assignments

- Due Mondays at 08:00
- Graded the day after (0-2 points)
- Y… n the n… oints
- A… be g… .
- Assignments will be handed in using CodeGrade.
- **Work individually**

Many exercises depend on the previous ones. Do not fall behind

Week	Date	Activity
Week 06	2/2 08:00	Assignment 1 deadline
Week 07	9/2 08:00	Assignment 2 deadline, Assignment 1 resubmission
Week 08	16/2 08:00	Assignment 3 deadline, Assignment 2 resubmission
Week 09	23/2 08:00	Assignment 4 deadline, Assignment 3 resubmission
Week 10		No assignment. Focus on the second-year project
Week 11	9/3 08:00	Assignment 5 deadline, Assignment 4 resubmission
Week 12	16/3 08:00	Assignment 6 deadline, Assignment 5 resubmission
Week 13	23/3 08:00	Assignment 7 deadline, Assignment 6 resubmission
Week 14		Easter Holiday

Week	Date	Activity
Week 15		No assignment. Focus on the second-year project
Week 16	13/4 08:00	Assignment 8 deadline, Assignment 7 resubmission
Week 17	20/4 08:00	Assignment 9 deadline, Assignment 8 resubmission
Week 18	27/4 08:00	Assignment 10 deadline, Assignment 9 resubmission
Week 19	4/5 08:00	Assignment 10 resubmission
Week 20	11/5 08:00	Final resubmission attempt **You may only resubmit assignments that you have previously made a serious attempt on and where an extension has been granted by the TAs.**
Week 22	28/5 09:00-14:00	Exam

Plagiarism

- You work on the assignments **individually**
- Do not copy code from other students!
- We will r… make sure that you …
- Using co… or ChatGPT and pres… rism.
- Plagiaris… exam, suspension for one or more semesters, or in some cases even expulsion.

Discuss high-level ideas and implementation strategies with your peers

NEVER SHARE CODE

Examination

- Five hour open-book exam
- You may use the book and physical notes that you bring
- You may not use the internet other than LearnIT
- You will need to hand in functional software that follows a strict specification
- We will give you plenty of old exams
- Mandatory assignments dramatically improve results. Take them seriously.

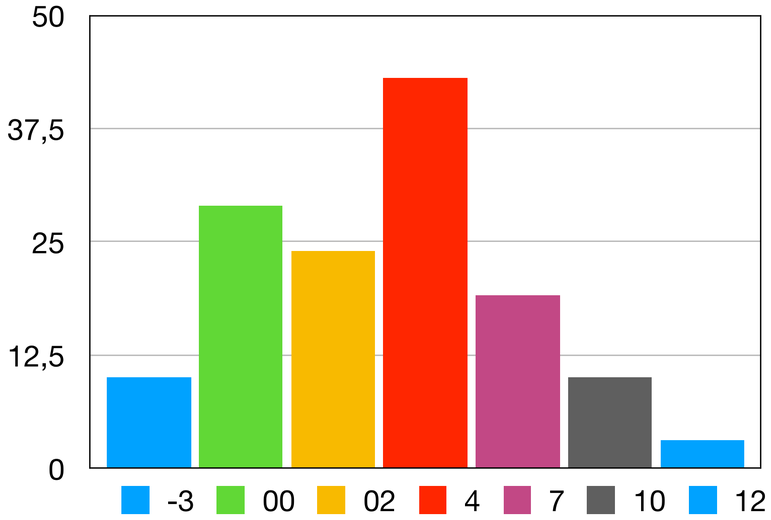
Examination

- Five hour open-book exam
- You may use the book and physical notes that you bri
- You ma nan LearnIT
- You will ftware that follows
- We will give you plenty of old exams
- Mandatory assignments dramatically improve results. Take them seriously.

We will use WiseFlow for the exam

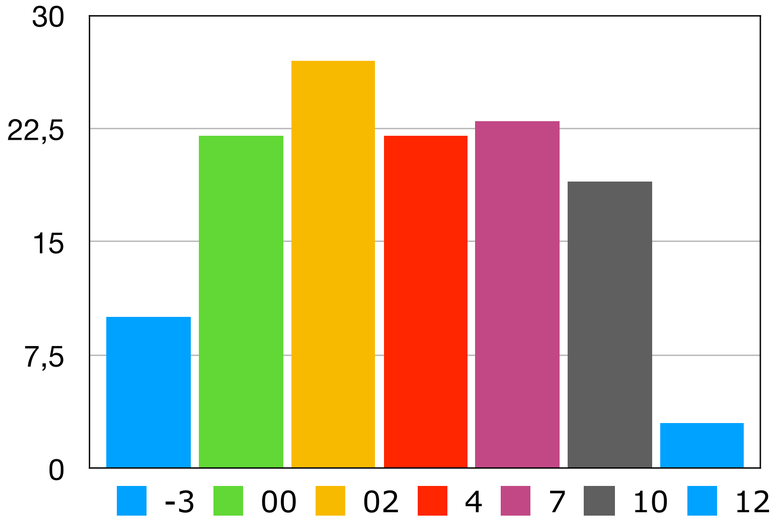
IT UNIVERSITY OF COPENHAGEN

Results 2020



Note: Group Assignments

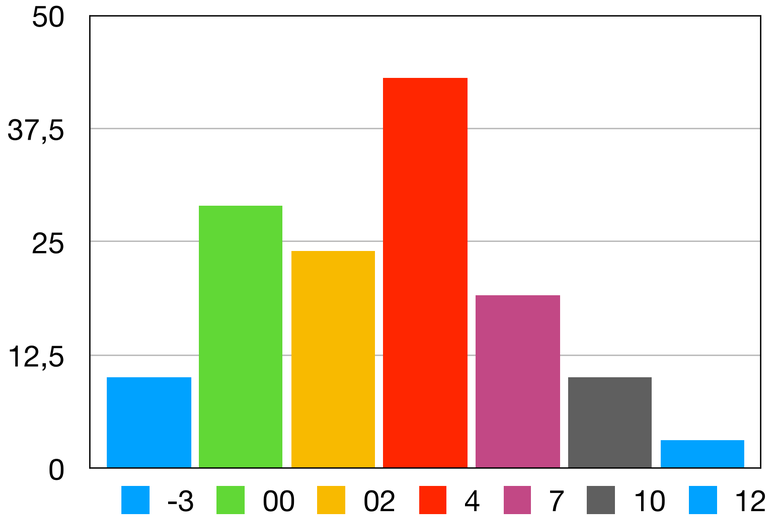
Results 2021



Note: Group Assignments

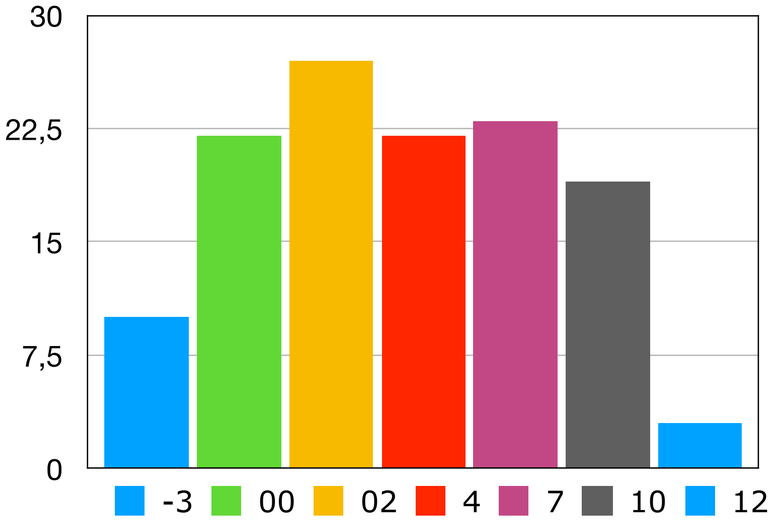
IT UNIVERSITY OF COPENHAGEN

Results 2020



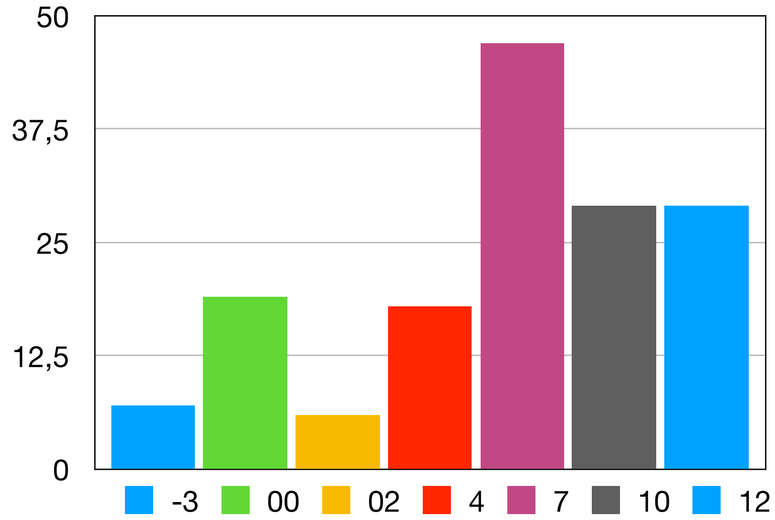
Note: Group Assignments

Results 2021



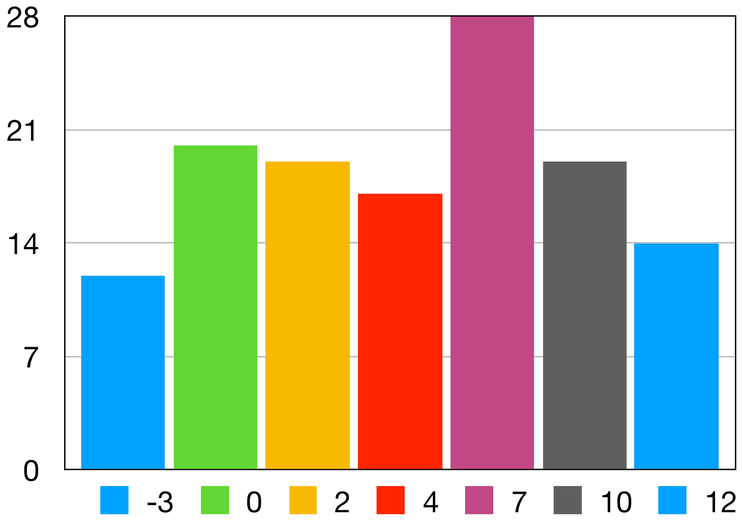
Note: Group Assignments

Results 2022



Note: Individual Assignments

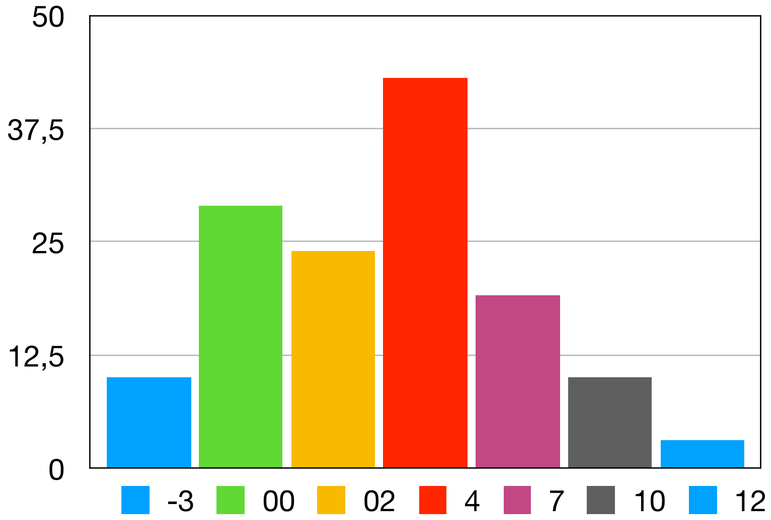
Results 2023



Note: Individual Assignments

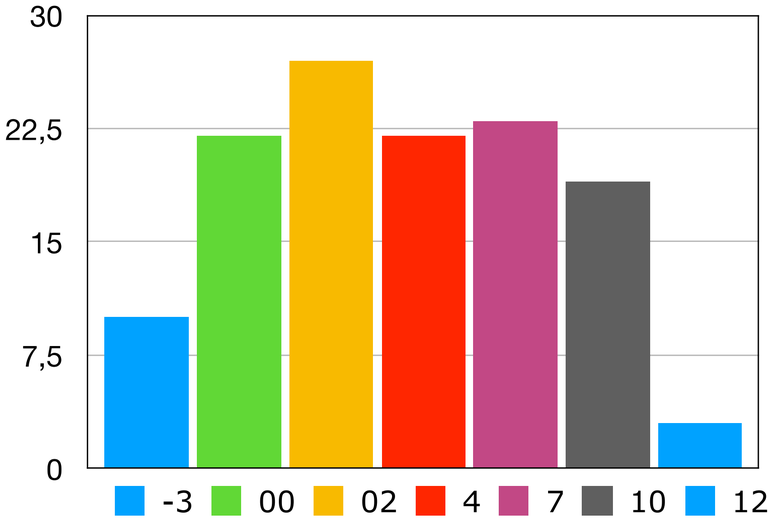
IT UNIVERSITY OF COPENHAGEN

Results 2020



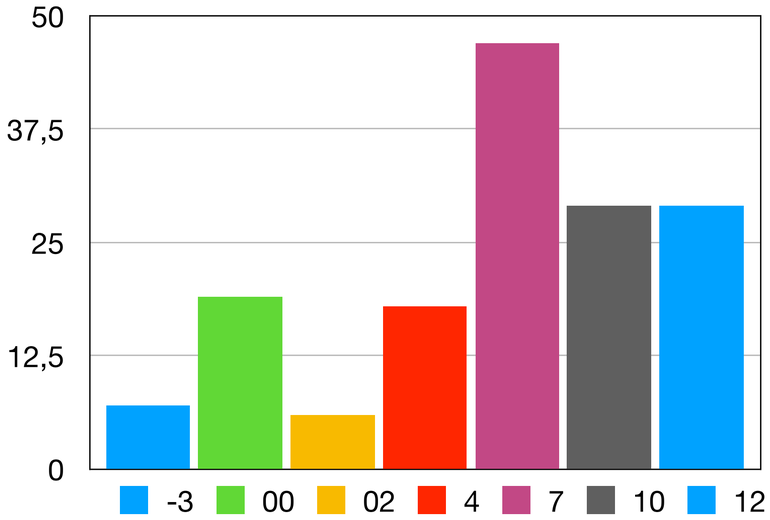
Note: Group Assignments

Results 2021



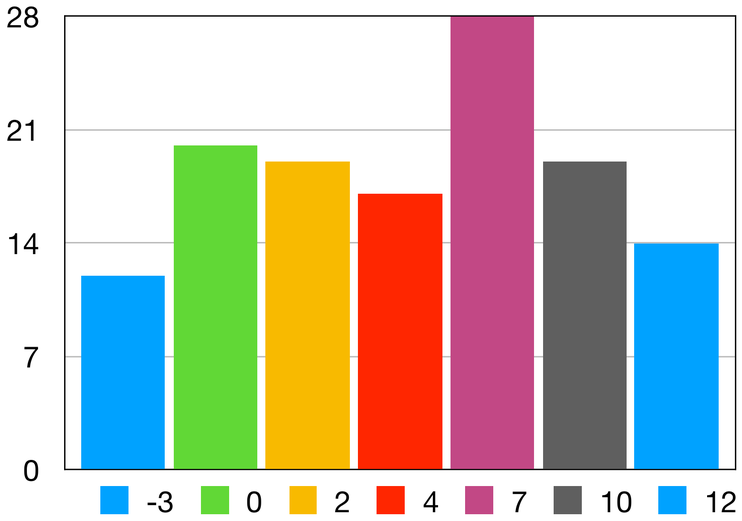
Note: Group Assignments

Results 2022



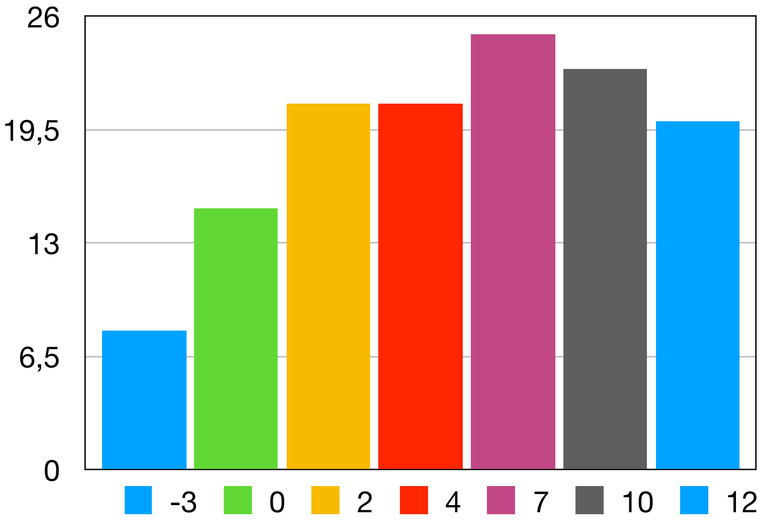
Note: Individual Assignments

Results 2023



Note: Individual Assignments

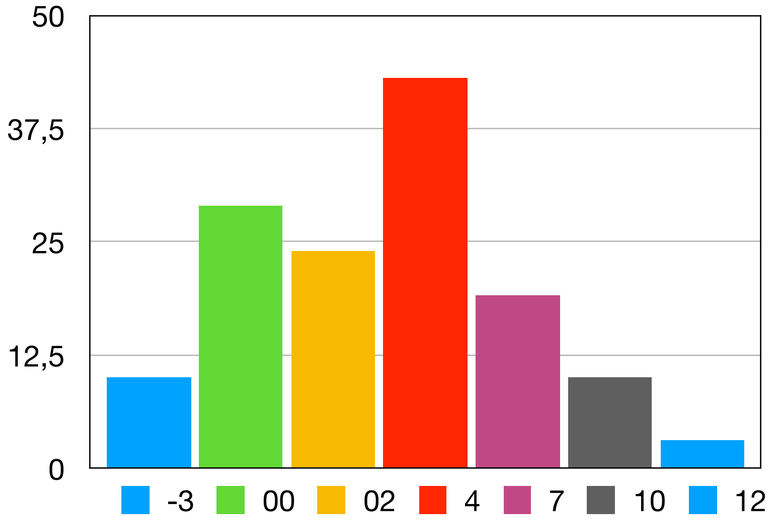
Results 2024



Note: Individual Assignments
Extra catch-up course

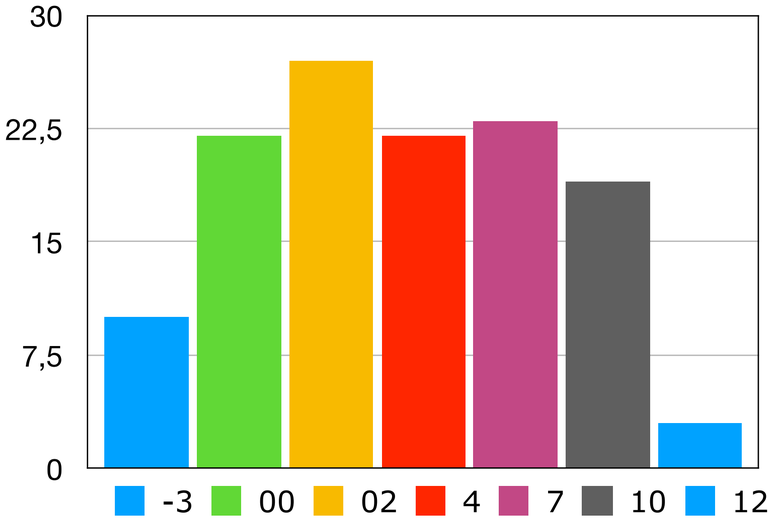
IT UNIVERSITY OF COPENHAGEN

Results 2020



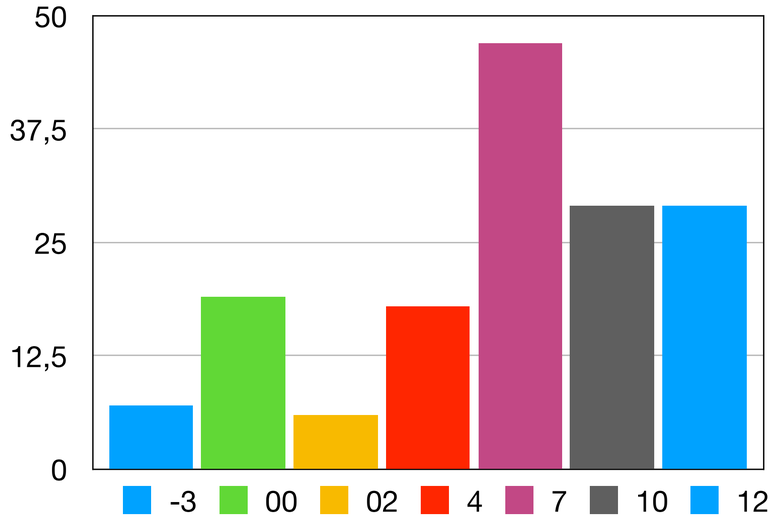
Note: Group Assignments

Results 2021



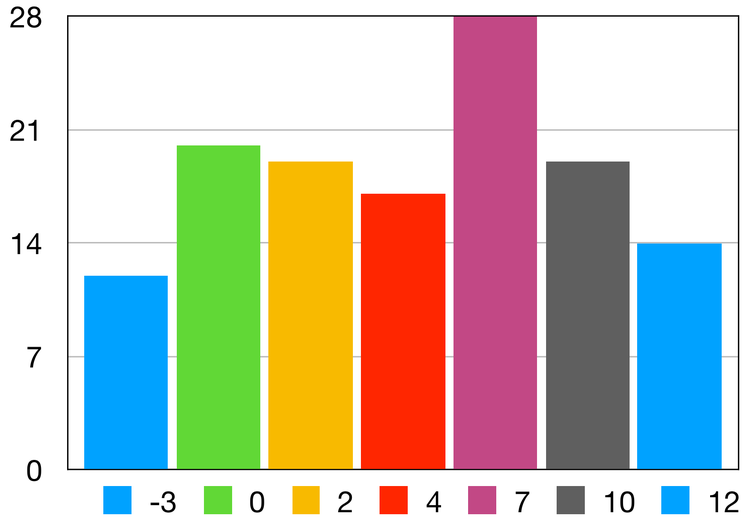
Note: Group Assignments

Results 2022



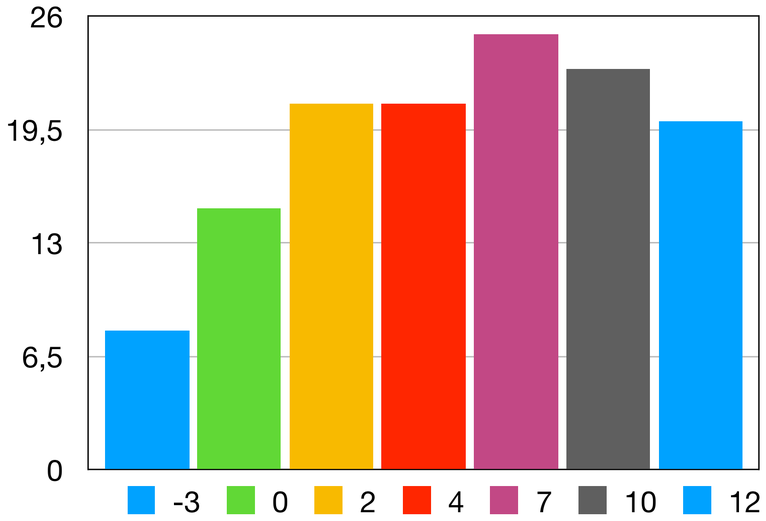
Note: Individual Assignments

Results 2023



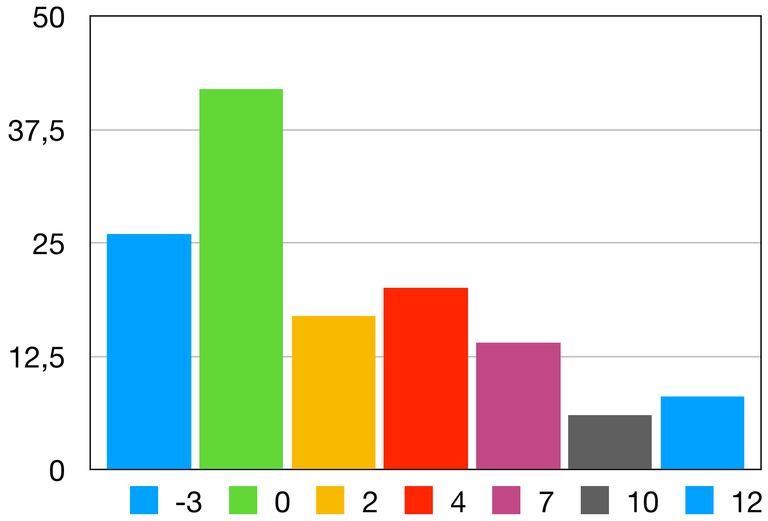
Note: Individual Assignments

Results 2024



Note: Individual Assignments
Extra catch-up course

Results 2025



Note: Individual Assignments
Scrabble Project removed
Exercise sessions less used
Very polished hand-ins

The use of AI

AI tools are revolutionary and knowing how to use them effectively will be an important skill for you as a developer

The use of AI

AI tools are revolutionary and knowing how to use them effectively will be an important skill for you as a developer

However

The use of AI

AI tools are revolutionary and knowing how to use them effectively will be an important skill for you as a developer

However

The foundation has to come first

The use of AI

This is a introductory course in functional programming and you develop core skills like

- Problem solving
- Reasoning about programs
- Translate ideas into correct code
- Learning a new paradigm

The use of AI

This is a introductory course in functional programming and you develop core skills like

- Problem solving
- Reasoning about programs
- Translate ideas into correct code
- Learning a new paradigm

The programs we work with here are small and constrained and you may very well struggle, but they are small compared to industry and easily solvable by AI

The use of AI

This is a introductory course in functional programming

The value in the exercises lies not in the code itself, but the thinking required to produce it.

Having your code auto-generated for you bypasses the learning process entirely

constrained and you may very well struggle, but they are small compared to industry and easily solvable by AI

The use of AI

For this course, any use of code-generating AI tools for the mandatory hand-ins (green, yellow, or red) or the exam will be considered **academic misconduct**.

The use of AI

For this course, any use of code-generating AI tools for the mandatory hand-ins (green, yellow, or red) or the exam will be considered **academic misconduct**.

You may use basic IDE functionality like basic IntelliSense

The use of AI

For this course, any use of code-generating AI tools for the mandatory hand-ins (green, yellow, or red) or the exam will be considered **academic misconduct**.

You may use basic IDE functionality like basic IntelliSense

You may use AI tools to ask questions about general concepts, terminology, or high-level explanations, provided that you do not ask them to write, modify, or complete code for you in any form

The use of AI

So what should you do

- Make use of your friendly and motivated TAs
- Go to the exercise sessions
- Go to the live-coding sessions
- Ask me or Patrick
- Make use of the Q&A channel
- Do Kattis exercises

The use of AI

After this course you should be in a state where AI provides a useful tool for you, but now is not the time

Introduction to FP

These slides are based on slides by
Michael R. Hansen.

Lecture outline

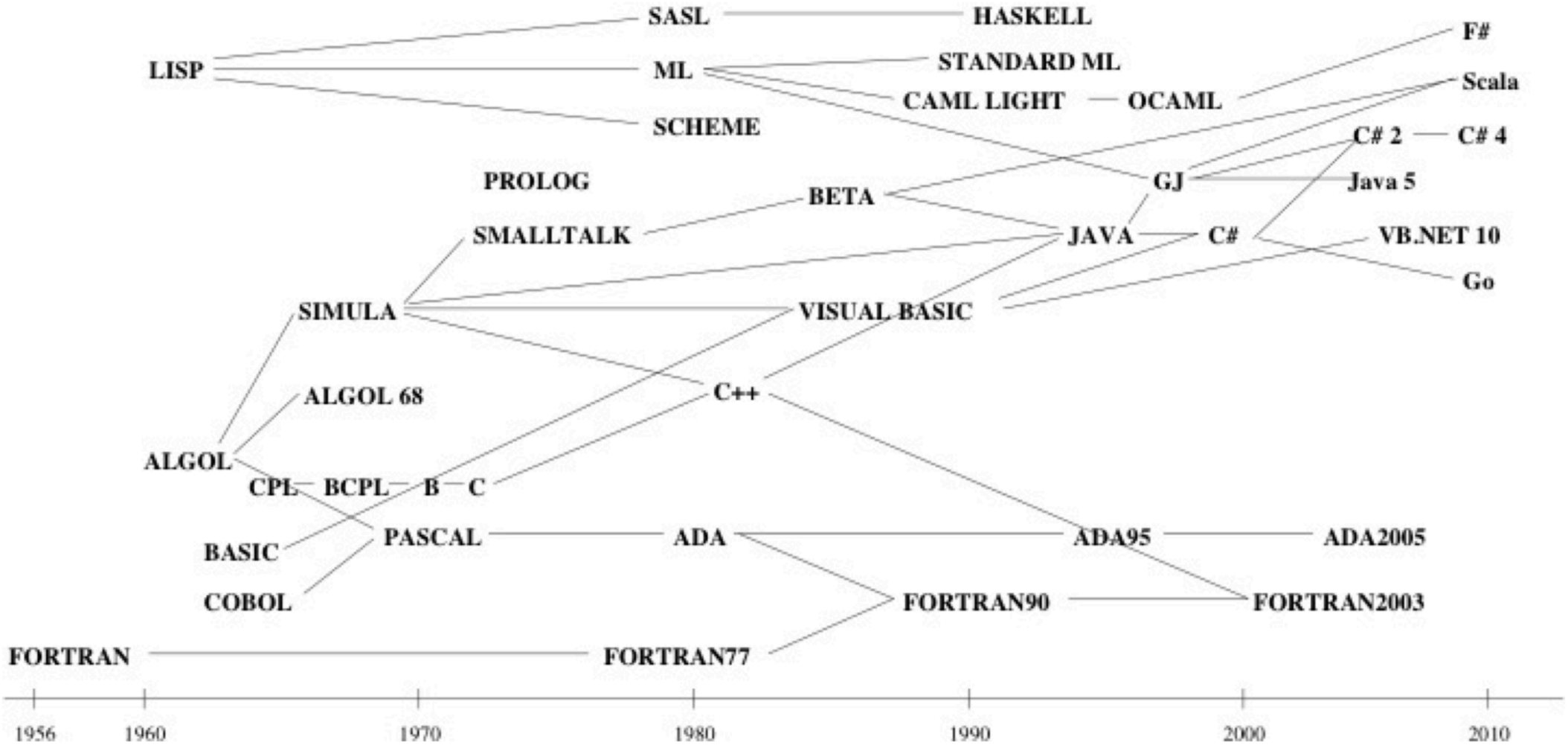
1. Introduction
2. Getting started with F#
3. Brief introduction to lists

Part 1

Introduction

- Historic perspective
- Imperative models
- Object-oriented models
- Declarative models
- Functional Programming background

Historic perspective



Imperative programming

Imperative programs are modelled using state and state-changing operations

```
fact(i):
  r := 1;
  while i > 0 do
     r := r*i;
     i := i-1;
  od
  return r;
```

Imperative models describe **how** a solution is obtained

OO programming

- Add structure to imperative models
- Objects are modelled using state and an interface of state-changing operations
- OO models include collections of object that communicate over these interfaces

Object-oriented models describe **how** a solution is obtained

Declarative programs

In declarative models, the focus is on **what** a solution is.

In functional programming, a program is expressed as a mathematical function:

```
fact(0) = 1
fact(x) = x * fact(x - 1) [if x > 0]
```

Running a functional program means evaluating a function.

Benefits of FP

- Fast prototyping
- Easy to reason about smaller features (functions) to build lager features (application of functions)
- Relatively easy to parallelise
- Functions can be composed easily
- Smaller, easier to read programs

Theoretical foundations

Alonzo Church
1903-1995

Introduced the λ-calculus around 1930

`f(x) = x + 2` is written as `λx. x + 2`

Forms the foundational underpinnings of all functional programming languages

Functional languages

- Meta Language (ML) was originally designed for theorem proving

  Gordon, Milner, Wadsworth (1977)

- High quality compilers, e.g. Standard ML of New Jersey and Moscow ML, based on a formal semantics

  Milner, Tofte, Harper, MacQueen 1990 & 1997

- Functional languages are widely used e.g. for AI, the finance industry, and compilers

A major goal

Teach abstraction (not concrete languages)

- Modelling
- Design
- Programming

Let us solve programs in a succinct, elegant and *understandable* manner

A major goal

Once mastered, functional programming paradigms are highly useful even when programming using other paradigms

Mastering functional programming will make you a better programmer

A skilled programmer is not constrained by languages nor paradigms

Part 2

Getting started with F#

- The interactive environment
- Declarations
- Recursion
- Types

F# supports

- Functions as first-class citizens
- Structured data (lists, trees, …)
- Type inference and polymorphism
- Imperative and object-oriented programming (mutable data, loops arrays)

F# supports

In a given programming language design, a first-class citizen is an entity which supports all the operations generally available to other entities. These operations typically include being passed as an argument, returned from a function, and assigned to a variable.

Wikipedia

F# supports

- Functions as first-class citizens
-
-
- programming (mutable data, loops arrays)

Imperative features will be used very sparingly in this course, but they do have their place

The REPL

F# has an interactive environment to help develop and debug programs

```
> 2 * 3 + 4;;
val it : int = 10
```

Input to F#

Response from F#

- **val** indicates that a value is computed
- The integer **10** is the computed value
- **int** is the type of the computed value
- The identifier **it** names the last result

The REPL

F# has an interactive environment to help develop and debug programs

```
> 2 * 3 +
val it : i
```

Input to F#

onse from F#

We write

`it ↦ 10`

as `it` is bound to `10`

- **val** indic
- The integ
- **int** is the type of the computed value
- The identifier **it** names the last result

Value declarations

We can bind values to our own identifiers

```
> let price = 25 * 5;;
val price : int = 125
```

Input to F#

Response from F#

The effect of this declaration is a binding

price ↦ 125

Environments

A collection of bindings is called an *environment*

```
> let price = 25 * 5;;
val price : int = 125
> let newPrice = price * 2;;
val newPrice : int = 250
> newPrice > 500;;
val it : bool = false
```

The above produces the environment
[price ↦ 125; newPrice ↦ 250; it ↦ false]

Function declarations

Functions are declared in a similar fashion to values

```
> let circleArea r = System.Math.PI * r * r;;
val circleArea : float -> float
```

- System.Math is a program library
- PI is an identifier of type float in System.Math
- The function type is `float -> float` (*<argument type>* `->` *<return type>*)

Function application

```
> circleArea 1.0;
val it : float = 3.141592654

> circleArea 3.2;
val it : float = 32.16990877
```

Primitive types

F# supports the standard primitive types

- int (`1, 42, 123, -5, …`)
- float (`1.0, 42.0, 1.23, -5.34, …`)
- bool (`true, false`)
- string (`“Hello \”World\””`)
- char (`‘a’, ‘B’, ‘\n’, …`)
- …

Anonymous functions

Functions can be given inline using a *function expression:*

```
> fun r -> System.Math.PI * r * r;;
val it : float -> float
```

This expression is equivalent to the function `circleArea` we defined earlier:

```
> let circleArea r =
    System.Math.PI * r * r;;
```

Anonymous functions

Function expression with patterns

```
> function
    | true -> 0
    | false -> 1
val it : bool -> int

> it false
val it : int = 1
```

A functional expression that converts booleans to integers

Anonymous functions

Function expression with patterns

```
> function
    | 0 -> true
    | 1 -> false
val it : int -> bool

> it 0
val it : bool = true
```

A functional expression that converts integers to booleans

Anonymous functions

Function expression with patterns

```
> function
    | 0 -> true
    | 1 -> false
val it : int -> bool

> it 0
val it : bool = true
```

A functional expression that converts integers to booleans

This program is problematic. Why?

Anonymous functions

Function expression with patterns

```
> function
    | 0 -> true
    | 1 -> false
val it : int -> bool

> it 0
val it : bool = true
```

A functional expression that converts integers to booleans

There are more integers than 0 and 1

Anonymous functions

Function expression with patterns

```
> function
    | 0 -> true
    | _ -> false
val it : int -> bool

> it -578
val it : bool = false
```

A functional expression that converts integers to booleans

Recursive functions

Recall mathematical definition of factorial:

```
fact(0) = 1
fact(x) = x * fact(x - 1) [if x > 0]
```

Recursive functions are declared in F# using the **rec** keyword:

```
> let rec fact =
    function
    | 0 -> 1
    | x -> x * fact(x - 1);;
val fact : int -> int
```

Evaluation

```
> let rec fact =
    function
    | 0 -> 1
    | x -> x * fact(x - 1);;

fact 3
```

Evaluation

```
> let rec fact =
    function
    | 0 -> 1
    | x -> x * fact(x - 1);;
```

```
fact 3 ⇝ 3 * fact (3 - 1)
```

Evaluation

```
> let rec fact =
    function
    | 0 -> 1
    | x -> x * fact(x - 1);;
```

```
fact 3 ⇝ 3 * fact (3 - 1) ⇝
3 * fact 2
```

Evaluation

```
> let rec fact =
    function
    | 0 -> 1
    | x -> x * fact(x - 1);;
```

```
fact 3 ⇝ 3 * fact (3 - 1) ⇝
3 * fact 2 ⇝ 3 * 2 * fact(2 - 1)
```

Evaluation

```
> let rec fact =
    function
    | 0 -> 1
    | x -> x * fact(x - 1);;
```

```
fact 3 ⇝ 3 * fact (3 - 1) ⇝
3 * fact 2 ⇝ 3 * 2 * fact(2 - 1) ⇝
3 * 2 * fact 1
```

Evaluation

```
> let rec fact =
    function
    | 0 -> 1
    | x -> x * fact(x - 1);;
```

```
fact 3 ⇝ 3 * fact (3 - 1) ⇝
3 * fact 2 ⇝ 3 * 2 * fact(2 - 1) ⇝
3 * 2 * fact 1 ⇝
3 * 2 * 1 * fact (1 - 1)
```

Evaluation

```
> let rec fact =
    function
    | 0 -> 1
    | x -> x * fact(x - 1);;
```

```
fact 3 ⇝ 3 * fact (3 - 1) ⇝
3 * fact 2 ⇝ 3 * 2 * fact(2 - 1) ⇝
3 * 2 * fact 1 ⇝
3 * 2 * 1 * fact (1 - 1) ⇝
3 * 2 * 1 * fact 0
```

Evaluation

```
> let rec fact =
    function
    | 0 -> 1
    | x -> x * fact(x - 1);;
```

```
fact 3 ⇝ 3 * fact (3 - 1) ⇝
3 * fact 2 ⇝ 3 * 2 * fact(2 - 1) ⇝
3 * 2 * fact 1 ⇝
3 * 2 * 1 * fact (1 - 1) ⇝
3 * 2 * 1 * fact 0 ⇝
3 * 2 * 1 * 1
```

Evaluation

```
> let rec fact =
    function
    | 0 -> 1
    | x -> x * fact(x - 1);;
```

```
fact 3 ⇝ 3 * fact (3 - 1) ⇝
3 * fact 2 ⇝ 3 * 2 * fact(2 - 1) ⇝
3 * 2 * fact 1 ⇝
3 * 2 * 1 * fact (1 - 1) ⇝
3 * 2 * 1 * fact 0 ⇝
3 * 2 * 1 * 1 ⇝ 6
```

Pairs

Pairs consist of two values

```
> (5, 3);;
val it : int * int = (5, 3)

> (System.Math.PI, false)
val it : float * bool = (3.141592654, false)

> (“A function!!!”, fact)
val it : string * (int -> int) =
…
```

Pairs

```
> (5,
val it

> (Sys
val it                                    lse)

> (“A function!!!”, fact)
val it : string * (int -> int) =
…
```

Functions are first-class citizens.

They can be used as data just as well as any other value

There is conceptually no difference between providing an string as an argument or a function as an argument, as demonstrated here

If-expressions

If-expressions have the form

```
if b then e1 else e2
```

Evaluation of if-expressions:

```
if true then e1 else e2 ⇝ e1
if false then e1 else e2 ⇝ e2
```

Example

```
> 1 + (if 1 > 1 then 1 else 2);;
val it : int = 3
```

If-expressions

A comparison

```
let rec fact =
function
| 0 -> 1
| x -> x * fact(x - 1)
```

```
let rec fact x =
    if x = 0 then
        1
    else
        x * fact(x - 1)
```

If-expressions

A comparison

```
let rec fact =
function
| 0 -> 1
| x -> x * fact(x - 1)
```

```
let rec fact x =
    if x = 0 then
        1
    else
        x * fact(x - 1)
```

If-expressions naturally have their uses, but pattern matching is often cleaner

Currying functions

How many arguments does the following function take?

```
let f (x, y) = x + y
```

Currying functions

How many arguments does the following function take?

```
let f (x, y) = x + y
```

It takes one argument!!!
It takes one pair of two integers

```
f : (int * int) -> int
```

Currying functions

How many arguments does the following function take?

```
let f (x, y) = x + y
```

It takes one argument!!!
It takes one pair of two integers

```
f : (int * int) -> int
```

Why do we care?

Currying functions

How many arguments does the following function take?

```
let f x y = x + y
```

Currying functions

How many arguments does the following function take?

```
let f x y = x + y
```

It takes two arguments (sort of)

```
f : int -> int -> int
```

Currying functions

How many arguments does the following function take?

```
let f x y = x + y
```

```
f 5 ⇝ ???
```

Currying functions

How many arguments does the following function take?

```
let f x y = x + y
```

```
f 5 ⇝ fun y -> 5 + y
```

Currying functions

How many arguments does the following function take?

```
let f x y = x + y
```

```
f 5 ⇝ fun y -> 5 + y
```

This is called partial application

Currying functions

This is important because

- Partial application allows us to specialise a function for future use
- Immensely powerful with first class functions

Currying functions

This is important because

- There are always edge-cases, but (with very few exceptions) functions should be curried rather than taking one giant tuple as an argument.
- s

Doing so is simply too restrictive

Type inference

What is the type of this function?

```
let rec pow =
  function
  | (x, 0) -> 1.0
  | (x, n) -> x * pow (x, n - 1)
```

Type inference

What is the type of this function?

```
let rec pow =
  function
  | (x, 0) -> 1.0
  | (x, n) -> x * pow (x, n - 1)
```

- `pow` must have type `t1 * t2 -> t3`
  - t3 must be float (1.0 is float)
  - t2 must be int (0 is int)
  - `x * pow (x, n - 1)` must have type float (t3 is float)

Type inference

What is the type of this function?

```
let rec pow =
  function
  | (x, 0) -> 1.0
  | (x, n) -> x * pow (x, n - 1)
```

- `pow` must have type `t1 * int -> float`
  - Since `x * pow (x, n - 1)` must have type float, we know `x` has type float
  - Therefore, `t1` is float

Type inference

What is the type of this function?

```
let rec pow =
  function
  | (_, 0) -> 1.0
```

Type inference is done automatically by F#

- t
  - Since x * pow (x, n - 1) must have type float, we know x has type float
  - Therefore, t1 is float

Kattis

- Used every semester at the LilleKat events that are held by the algorithms group
- Great way to hone your programming skills
- Thousands of exercises at all difficulty levels

IT UNIVERSITY OF COPENHAGEN

Hide

Leggja saman

LANGUAGES
en is

Arnar and Hannes are parking cars for party guests. When all party guests are seated and all cars have been parked they discuss how many cars they've parked.

Arnar tells Hannes how many cars he parked and Hannes tells Arnar how many cars he parked. Now they want to know how many cars they parked in total and ask for your help.

Parking by Egor Myznik, Unsplash

Input

The input is on two lines. The first line contains a single integer n ($2 \leq n \leq 1000$), the number of cars Arnar parked. The second line contains a single integer m ($2 \leq m \leq 1000$), the number of cars Hannes parked.

Output

Print a single integer, the total number of cars Arnar and Hannes parked.

Scoring

Group	Points	Constraints
1	100	No further constraints

Sample Input 1

```
4
3
```

Sample Output 1

```
7
```

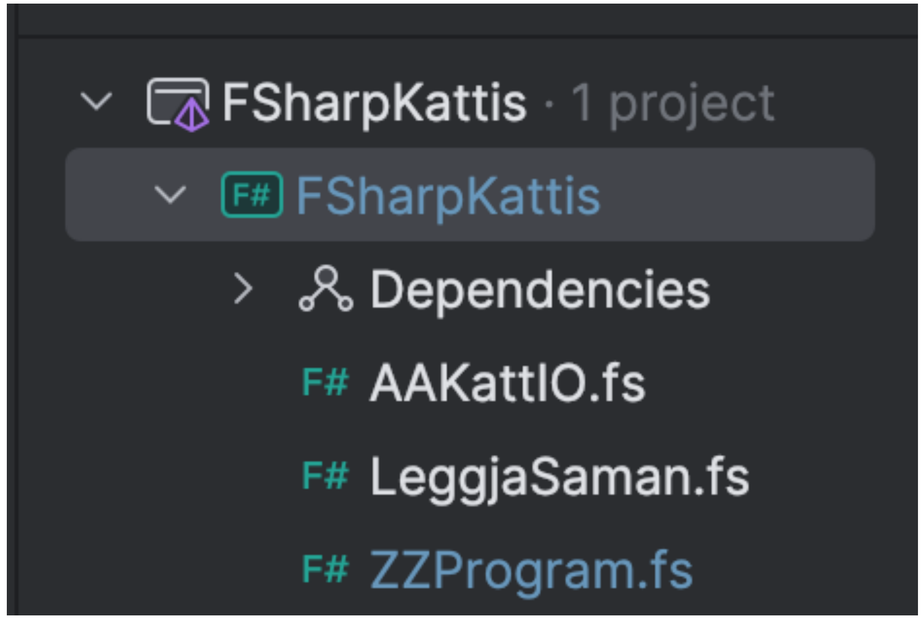
Each problem contains

- A colourful description
- Constraints on the input
- Description of the output
- A few simple, and by no means exhaustive, test cases

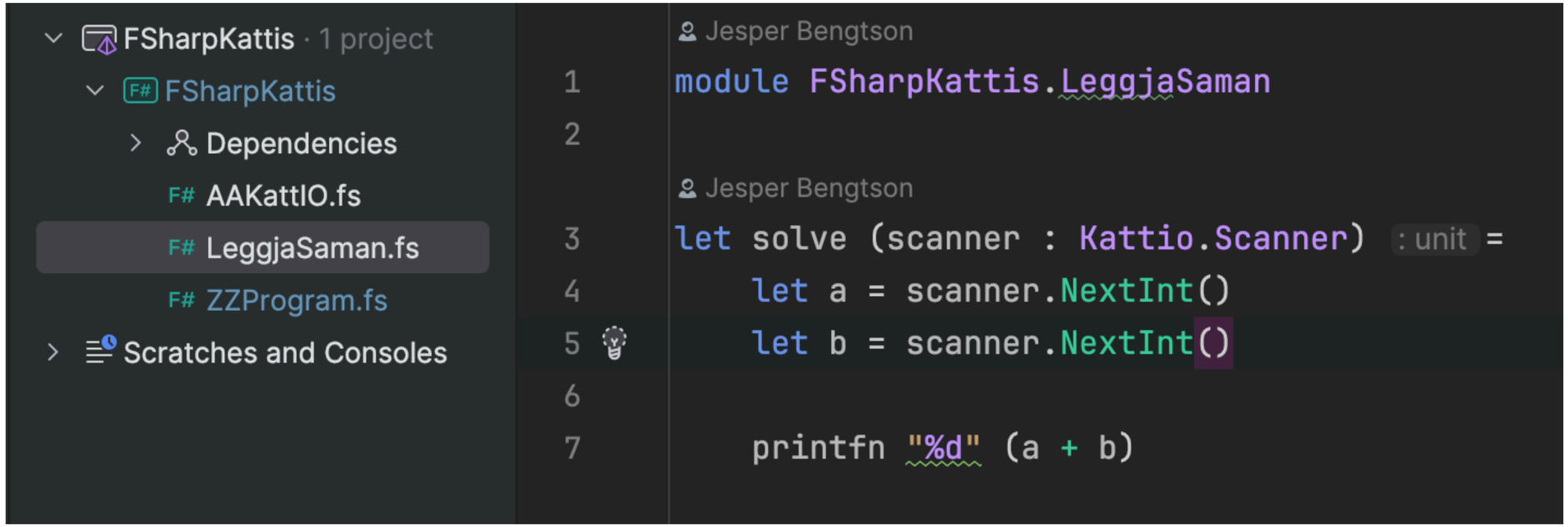
Kattis

We provide a project to help you with the exercises

- AAKattIO.fs lets you read from the terminal
- ZZProgram.fs starts the project
- The rest are your solutions that you can add one at a time, in this case it is LeggjaSamman.fs



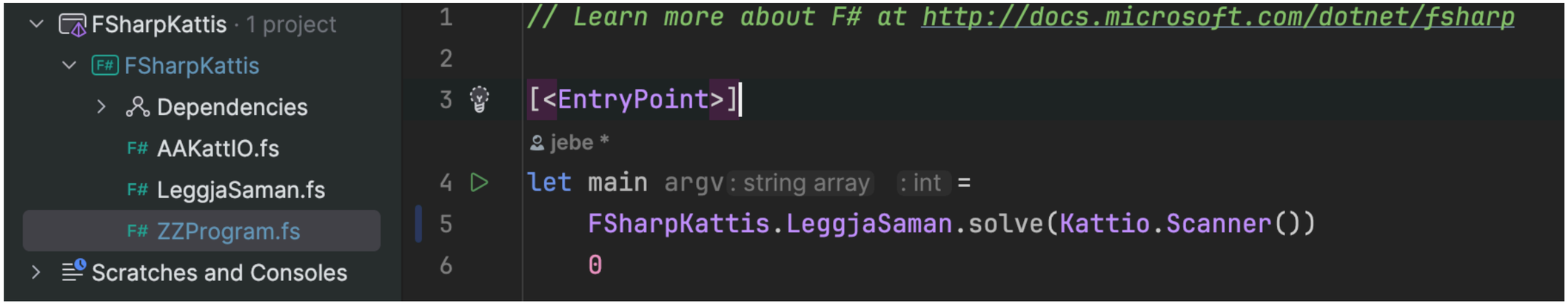
Kattis



Your solution should contain a solve function that takes a scanner as input and prints the result

printfn is powerful and works just as it does for C#

Kattis



The ZZProgram.fs files starts the program and calls the solve method with a new scanner.

When creating a new solution, just change LeggjaSamman to the name of your new solution

IT UNIVERSITY OF COPENHAGEN

‹ Hide

Leggja saman

LANGUAGES
en is

Arnar and Hannes are parking cars for party guests. When all party guests are seated and all cars have been parked they discuss how many cars they've parked.

Parking by Egor Myznik, Unsplash

Arnar tells Hannes how many cars he parked and Hannes tells Arnar how many cars he parked. Now they want to know how many cars they parked in total and ask for your help.

Input

The input is on two lines. The first line contains a single integer n ($2 \leq n \leq 1000$), the number of cars Arnar parked. The second line contains a single integer m ($2 \leq m \leq 1000$), the number of cars Hannes parked.

Output

Print a single integer, the total number of cars Arnar and Hannes parked.

Scoring

Group	Points	Constraints
1	100	No further constraints

Sample Input 1

Sample Output 1

Edit & Submit | Metadata | My Submissions | Hide ›

AAKattIO.fs × | LeggjaSaman.fs × | ZZProgram.fs × | F# | + New

```
1  module FSharpKattis.LeggjaSaman
2
3  let solve (scanner : Kattio.Scanner) =
4      let a = scanner.NextInt()
5      let b = scanner.NextInt()
6
7      printfn "%d" (a + b)
```

Upload your solution along with the IO and Program files. They are named so that they compile in the right order. Kattis did not want to figure out the order themselves, which is understandable.

Submit

IT UNIVERSITY OF COPENHAGEN

Submission 15303972

Edit and resubmit

DATE	PROBLEM	JUDGEMENT	RUNTIME	LANGUAGE	TEST CASES
06:10:00	Leggja saman	Accepted (100)	0.08 s	F#	24/24

Test groups | Submitted files | Submission history

Test Groups

SAMPLE	ACCEPTED (0/0)
SECRET	ACCEPTED (100/100)
GROUP 1	ACCEPTED (100/100)

Kattis

For this week we recommend

- Hello World
- Hipp Hipp
- Reduplication
- Stuck In A Time Loop
- N-Sum

Questions?